Greedy

Materia	Algoritmi
Anno	Secondo Anno
Data	@November 26, 2024
Studiato	Done

Alcuni problemi di ottimizzazione sono più semplici da trattare in virtù della loro natura e della loro impostazione. Infatti possiamo **individuare una caratteristica del problema che evita di considerare ogni soluzione possibile e stabilire la migliore**.
Per poter mettere in evidenza che caratteristica deve avere tale problema, analizziamo nuovamente il problema dello zaino.

Il problema dello zaino ha un parametro k che indica la capienza massima dello zaino e un insieme di elementi $A = \{ a_1, a_2, ..., a_n \}$ che possiamo mettere nello zaino. Associamo ad ogni elemento a_i un peso $P(a_i)$ e un altro numero $V(a_i)$ che rappresenta il suo valore. L'obiettivo era trovare un sottoinsieme $S \subseteq A$ che **massimizzi il valore complessivo degli oggetti presenti nello zaino**, ossia che massimizzi $\sum_{a \in S} V(a)$, con il vincolo che $\sum_{a \in S} P(a) \leq k$.

Supponiamo adesso di avere una formulazione leggermente diversa del problema. Se ogni elemento ha valore $v(a) = 1$, a questo punto la funzione che vogliamo massimizzare diventa la **cardinalità di** S, in quanto:

$$\sum_{a \in S} V(a) = \sum_{a \in S} 1 = |S|$$

Quindi più oggetti portiamo più guadagniamo. Possiamo allora usare una strategia che costruisce la soluzione ottima passo per passo. Intuitivamente ad ogni passo scegliamo l'elemento con peso minore.

Questa strategia prende il nome di **scelta greedy**, perché ad ogni passo prendiamo la **scelta ottima localmente**. Molto spesso è necessario dimostrare che

ogni scelta ottima locale permette di ottenere una scelta ottima globale.
Formalmente, la nostra scelta diventa:

$$a_w : P(a_w) = \min\{\, P(a_j) : a_j \in A \,\}$$

Dimostriamo che, se S è una soluzione ottima, con $S \subseteq A$, allora $a_w \in S$.

Neghiamo la tesi, e supponiamo che $a_w \notin S$. Prendiamo allora:

$$a_\phi : P(a_\phi) = \min\{\, P(a_j) : a_j \in S \,\}$$

Quindi prendiamo a_ϕ come **il più piccolo elemento della soluzione ottima**. Allora consideriamo $S - \{\, a_\phi \,\}$, e considero:

$$S' = (S - \{\, a_\phi \,\}) \cup \{\, a_w \,\}$$

Poiché S per ipotesi è una soluzione ottima, si ha che:

$$k \geq \sum_{a \in S} P(a)$$

Possiamo dire che la somma dei pesi degli elementi in S' è uguale alla somma dei pesi degli elementi presenti in S meno il peso di a_ϕ più il peso di a_w:

$$\sum_{a \in S'} P(a) = \sum_{a \in S} P(a) - P(a_\phi) + P(a_w)$$

e notiamo che:

$$k \geq \sum_{a \in S} P(a) \geq \sum_{a \in S'} P(a)$$

Di conseguenza, S' è una soluzione ottima e la tesi è dimostrata, in quanto $a_w \in S'$.

Esempio: codifica di Huffman

La codifica di **Huffman** risolve il problema della compressione. La compressione consiste nell'utilizzare un formato differente per **ridurre lo spazio utilizzato per**

rappresentare la stessa informazione contenuta nei dati da comprimere. Vi sono due modi per comprimere:

- compressione **lossy**, che riduce il testo T in un testo T' ma **non è possibile recuperare l'informazione persa** durante il processo di compressione. Quello che si ricostruisce è un testo T'' che non è identico a T. Spesso tagliamo informazione che non riusciamo a percepire;
- compressione **lossless**, che comprime il testo T in un testo T' e da cui possiamo ritornare al testo T. Il testo T e il testo T' contengono le stesse informazioni.

Il problema di ottimizzazione è di minimizzare la rappresentazione T', ossia **ridurre al minimo il numero di bit**. L'algoritmo si potrebbe riapplicare su T' ma il testo che si ottiene ha la stessa dimensione del testo originale.

L'algoritmo di Huffman definisce il modo per ridurre il più possibile il numero di bit utilizzato per memorizzare un testo. Supponiamo di avere un alfabeto $\sum = \{ a, b, c, d, e, f \}$. Il testo T è una sequenza di 20 caratteri:

$$T = \text{aabcaedfccaabafaeafd}$$

Ad ogni carattere associamo un numero a 8 bit (come ASCII). Tuttavia noi utilizziamo solo 6 simboli. Possiamo quindi utilizzare 3 bit per i 6 elementi del nostro alfabeto $\sum$.
La codifica di Huffman si basa sull'ipotesi che nel testo originale ci siano caratteri che appaiano più frequentemente rispetto agli altri. Nel nostro esempio, abbiamo 60 bit per la rappresentazione del nostro testo. Se alcuni simboli compaiono più frequentemente, **possiamo assegnare codifiche più piccole** per ridurre lo spazio. Costruiamo la tabella delle frequenze assolute per ogni simbolo:

a	8
b	2
c	3
d	2
e	2
f	3

Il peso del nostro testo T è dato da:

$$\sum_{c \in \Sigma} f(c) \cdot l(c)$$

dove $l(c)$ rappresenta la lunghezza della codifica di c.

Se assegniamo codifiche in maniera random, potrebbero verificarsi degli errori di interpretazione della sequenza di bit. Supponiamo di assegnare le seguenti codifiche:

a	11
b	01001
c	1001
d	00111
e	0111
f	011

e di avere la sequenza 0111100111. Allora possiamo decodificarlo in due modi diversi: eca oppure fad. Pertanto la codifica deve evitare che ci siano delle ambiguità tra i codici.

Generalmente si utilizzano i **codici prefissi**. Si dice che **un set di codici è prefisso se un codice associato ad un carattere non è mai il prefisso di un codice associato ad un altro carattere**. Nel nostro caso, il codice della c è il prefisso del codice della d, e il codice della f è il prefisso del codice della e.

Una possibile codifica diventa:

a	00
b	0110
c	111
d	0101
e	1101
f	101

E di conseguenza il testo T' ha peso:

$$2 \cdot 8 + 4 \cdot 2 + 3 \cdot 3 + 4 \cdot 2 + 4 \cdot 2 + 3 \cdot 3 = 58$$

In questo modo abbiamo una rappresentazione che risparmia due bit.

Per un set di caratteri ci sono molteplici codifiche. Pertanto ci chiediamo **qual è la codifica migliore**.

Possiamo rappresentare tale codifica come un **albero binario**, i cui rami presentano un peso corrispondente al valore possibile del bit e le cui foglie corrispondono ai caratteri. In tale albero:

- **dobbiamo evitare che una codifica sia radice di un sottoalbero che ha come foglia un'altra codifica**. Se ciò accade, **la codifica di un carattere è un prefisso di un'altra**;
- **dobbiamo evitare di avere nodi con un solo figlio**, in quanto i figli possono essere collassati verso l'alto e avere una codifica migliore.

Nella nostra codifica, possiamo migliorare utilizzando tale albero:

a	00
b	011
c	111
d	010
e	110
f	10

E abbiamo ben 49 bit.

Dobbiamo quindi **capire qual è l'albero migliore**.

Soluzione

Consideriamo ogni simbolo e la relativa frequenza:

$$\{a : 8, b : 2, c : 3, d : 2, e : 2, f : 3\}$$

L'idea è inserire i caratteri con frequenza minore in fondo.
Consideriamo due caratteri con frequenza minima

d, e e li fondiamo in un carattere non presente nell'alfabeto, ad esempio g, la cui frequenza è pari alla somma delle singole frequenze. Quindi otteniamo:

$$\{a:8, b:2, c:3, g:4, f:3\}$$

Creiamo quindi un albero con radice $g:4$ e figli $d:2, e:2$.
Consideriamo un'altra coppia di caratteri con frequenza minima. Uniamo $b:2$ e $c:3$ in $h:5$ e creiamo tale nodo nell'albero. Continuiamo, e abbiamo $g:4$ e $f:3$. A questo punto il nodo g diventa figlio di $i:7$.

Ad ogni passo quindi ogni carattere diventa un nodo con padre che deriva dall'unione dei due caratteri in un solo carattere con frequenza pari alla somma delle frequenze dei singoli caratteri. Otteniamo così una **codifica ottimale** per i caratteri, ma non è l'unica, perché possiamo scambiare tra di loro dei figli e quindi ottenere la stessa struttura ma codifiche diverse.

In questo caso, si ottiene una codifica:

a	0
b	100
c	101
d	1100
e	1101
f	111

Il contributo della a è 8, il contributo della b è 6, della c è 9, della d e della e è 8 e della f è 9. Sommando si ottiene 48.

Pseudocodice

Per avere una struttura dati che ci permette di selezionare ad ogni passo l'elemento più piccolo utilizziamo un **Min-Heap**.

```
Huffman(Sigma, f): // f = frequenza di ogni elemento
    N = {node(fc) | c in Sigma } //creiamo un insieme che ha le coppie lettera-freq
    n = Sigma.size
    Q = build-min-heap(N) //costruisce min-heap su N, usando come chiave la fre
```

```
    // dopo n-1 passi, otteniamo un alfabeto con un solo carattere, radice dell'albe
    for i = 1 to n-1 do
        a = extract-min(Q)
        b = extract-min(Q)
        c = New-Node(f(a)+f(b))
        left(c) = a
        right(c) = b
        insert(Q,c)
    return extract-min(Q) // restituisce la radice dell'albero costruito
```

Analizziamo:

- per la creazione di N impieghiamo un tempo $O(n)$;
- per la creazione del min-heap impieghiamo $O(n)$;
- il ciclo viene eseguito $n-1$ volte:
  - l'estrazione costa $O(\log n)$, eseguita due volte;
  - l'inserimento costa $O(\log n)$.

Quindi in totale abbiamo una complessità $O(n + n \log n) = O(n \log n)$.

Se utilizzassimo un array per gestire le frequenze, nel caso pessimo impiegheremo $O(n^2)$. Quindi il min-heap è una scelta migliore.

Creiamo ora una procedura per ottenere il codice del carattere x:

```
Get-code(x):
    S = Stack()
    p = Parent(x)
    while p != null do
        if x = left(p) then
            push(S, 0)
        else
            push(S,1)
        x = p
        p = parent(x)
```

```
    while S != empty do
        print pop(S)
```

La sua versione ricorsiva:

```
Get-code(x):
    if parent(x) = null then
        return
    p = parent(x)
    Get-code(p)
    if x = left(p) then
        print(0)
    else print(1)
```

Per la decodifica quindi partiamo dal basso. Ma per la codifica partiamo dall'alto e scendiamo scegliendo il ramo che corrisponde al bit osservato. Dopo ogni carattere trovato, ripartiamo dalla radice.